VIRCL · VIRTUALISATION · DONOVAN GLODT & YONA JERVIS

Snapshots & Templates

Concept, implementation and best practice

VMware vSphere / ESXi 8

Proxmox VE 9.1

AGENDA

How we'll cover it

Two topics

- **1 · Snapshots** — capturing a point in time
- **2 · Templates** — mass, repeatable deployment

Same flow for each

- What it **is** & how it **works**
- In **VMware**, then in **Proxmox**
- The key **distinctions**
- **Do's & Don'ts**

TOPIC 01

Snapshots

A point-in-time freeze of a VM you can roll back to — the safety net before any risky change.

Definition | How it works | VMware | Proxmox | Comparison | Do's & Don'ts

SNAPSHOTS · DEFINITION

What is a snapshot?

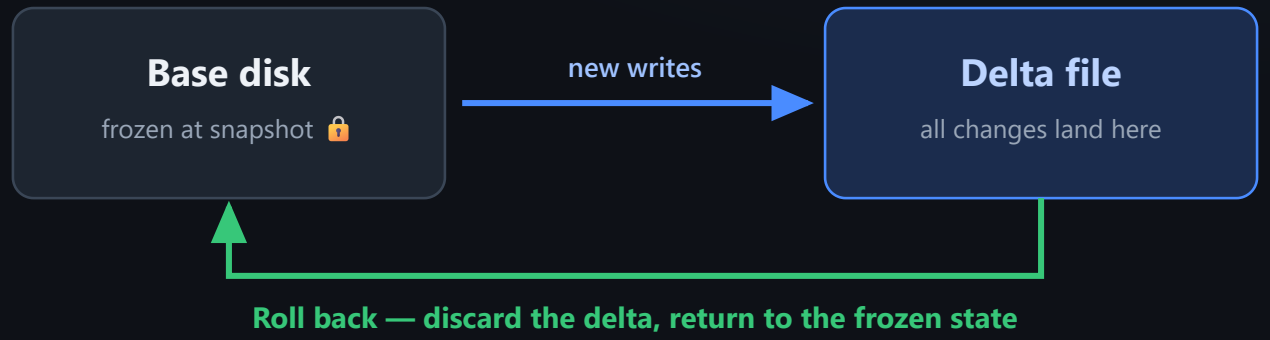
A snapshot preserves the state of a VM **at a specific point in time** — its virtual disk, its settings, and **optionally its memory** (the running state).

It works as a **delta / differencing layer** on top of the base disk: once taken, all new writes go into a separate delta file while the original disk is frozen — which is exactly what lets you roll back to the captured moment.

> **A snapshot is NOT a backup.** It depends on the base disk — if the base VM is lost or corrupted, the snapshot is worthless too.

SNAPSHOTS · HOW IT WORKS

The delta layer



Use it for

- Right before **patching, updates or config changes** you might need to undo

Keep it short-lived

- A long-running snapshot **keeps growing** and degrades VM performance

SNAPSHOTS · VMWARE

Snapshots in vSphere

VMWARE Take & revert

- Two routes: the VM's **action menu**, or the dedicated **Snapshot tab**
- Name it and choose to **save memory** too
- Revert → confirm → the test file we created is **gone**; the delta layer was simply dropped
- Managed centrally in the **vCenter Snapshot Manager**

VMWARE Delete ≠ discard

- **Delete** — commits that delta *back into* the base disk, then removes the delta (changes kept, chain shortened)
- **Delete All** — collapses the whole chain into the base disk
- **Consolidate** — cleans up orphaned delta files left by a failed delete **vCenter warns you**

SNAPSHOTS · PROXMOX

Snapshots in Proxmox

PROXMOX Take & rollback

- Handled **per-VM** under the VM's own **Snapshots tab** — no central manager like vSphere
- **Take Snapshot** → name → optionally tick **Include RAM** for the running state
- **Rollback** → confirm → file gone, VM exactly as it was

PROXMOX Storage caveats

- Snapshot support is **tied to the storage type**
- Works on **qcow2, LVM-thin, ZFS, Ceph**
- A disk on **plain LVM can't be snapshotted** key gotcha

SNAPSHOTS · COMPARISON

VMware vs Proxmox

	VMWARE VSPHERE	PROXMOX VE
Where	Central **Snapshot Manager** in vCenter	Built into the **VM panel** — no vCenter needed **simpler**
Storage	Snapshots **any** datastore type	Only on **snapshot-capable** storage; not plain LVM
Memory / RAM	Included **by default**	Captured **only if you tick the box**
Example size	**70.28 GB** (RAM included)	Smaller — RAM only when needed

> Trade-off in one line: Proxmox is **easier to reach** but **fussier about storage**; VMware snapshots anything but defaults to **bigger** snapshots.

SNAPSHOTS · BEST PRACTICE

Do's & Don'ts

✓ Do

- ✓ Take one **before any risky change** — patching, updates, config you might undo
- ✓ **Delete it again** once the change is confirmed good
- ✓ Include **memory/RAM only** when you genuinely need the running state

✕ Don't

- ✕ Treat a snapshot as a **backup** — it dies with the base disk
- ✕ Leave snapshots lying around for **more than a day or two** — the delta grows and degrades performance
- ✕ Tick **Include RAM** by reflex — you're just wasting storage

TOPIC 02

Templates

A master image you stamp out consistent VMs from — built for repeatable, generalised deployment.

Definition | Clone vs Template | VMware | Proxmox | When to use which | Do's & Don'ts

TEMPLATES · DEFINITION

What is a template?

A **master image** used as a baseline for deployment. The template is **not a VM itself** — it exists to stamp out new VMs with repeatability.

Templates rely on **generalisation**: the VM converted into a template has its unique identifiers stripped — **hostname, MAC address, SSH keys** — so every VM deployed from it gets its own identity and doesn't conflict with the others.

> **Generalisation** is the whole point — without it, ten VMs from one template all collide on the network sharing the same identity.

TEMPLATES · THE DISTINCTION

Clone vs Template

CLONE One-to-one copy

- An exact copy of a **single** VM
- **Full clone** — exact, independent copy
- **Linked clone** — shares the base disk with the source
- Right tool for a **one-off copy** — identity and all

TEMPLATE Mass deployment

- A **master image** for consistent, repeatable deployment
- Stays as an **untouched baseline** you stamp VMs from
- Built for **many** machines over time, generalised per clone

It comes down to need: a **one-time exact copy** (clone) vs a **consistent way to deploy many** of the same VM (template).

TEMPLATES · VMWARE

Templates in vSphere

VMWARE Convert & deploy

- Action menu → **Template** → **Convert to Template** (removes it from the VM list)
- Deploy → new VM → **Deploy from template** → pick compute & storage
- **Two-way:** Template → **Convert to Virtual Machine** any time — patch the master, then convert back

vs Proxmox: one-way

VMWARE Generalisation

- The **"Customize the operating system"** clone option is what makes deployments unique
- Backed by a **guest customisation spec**, it strips & regenerates hostname, SID/MAC, etc. per clone
- Left unchecked → deployed VMs **share the source's identity** our screenshots' caveat
- Direct equivalent of Proxmox's **Cloud-Init** step

TEMPLATES · PROXMOX

Templates in Proxmox

PROXMOX Convert & clone

- **More** → **Convert to template** (or `qm template <vmid>`)
- **One-way & permanent** — a template can't be started and can't convert back. Always keep the original VM or a clone **key difference**
- Deploy = **right-click** → **Clone**; now the **Mode** dropdown unlocks both clone types

PROXMOX Full vs Linked · Cloud-Init

- **Full clone** — independent copy, any storage, slower, full disk each time
- **Linked clone** — delta on the template's base, near-instant & space-efficient; needs snapshot-capable storage, blocks template deletion
- **Cloud-Init** (Linux) / sysprep (Windows) gives each clone its own IP, hostname & SSH keys on first boot

TEMPLATES · COMPARISON

When to use which

	VMWARE	PROXMOX
Conversion	**Two-way** — convert back to a VM anytime	**One-way & permanent** — keep the original
Deploy default	**Independent** VMs by default	Can deploy **linked** (shared base disk)
Generalisation	"Customize the OS" + guest spec	**Cloud-Init** / sysprep

Clone for a single exact copy *now* · **Template** for repeatable, generalised deployment *later* — ideally with Cloud-Init or guest customisation so each VM boots with its own identity.

TEMPLATES · BEST PRACTICE

Do's & Don'ts

✓ Do

- ✓ In Proxmox, **keep the original VM or a clone** — conversion is permanent
- ✓ Use **Cloud-Init / guest customisation** when deploying more than one
- ✓ Keep the template a **clean, known-good baseline**

✗ Don't

- ✗ Deploy multiple clones **without generalisation** — they'll collide on identity
- ✗ Rely on **linked clones on plain LVM** — needs snapshot-capable storage
- ✗ **Delete a template** while linked clones still depend on it

WRAP-UP

Key notes

Snapshots

A short-lived point-in-time safety net — **not a backup**. Take before risk, delete after.

Templates

A generalised master image for **mass, repeatable deployment** — clone is the one-off cousin.

Thank you — questions?